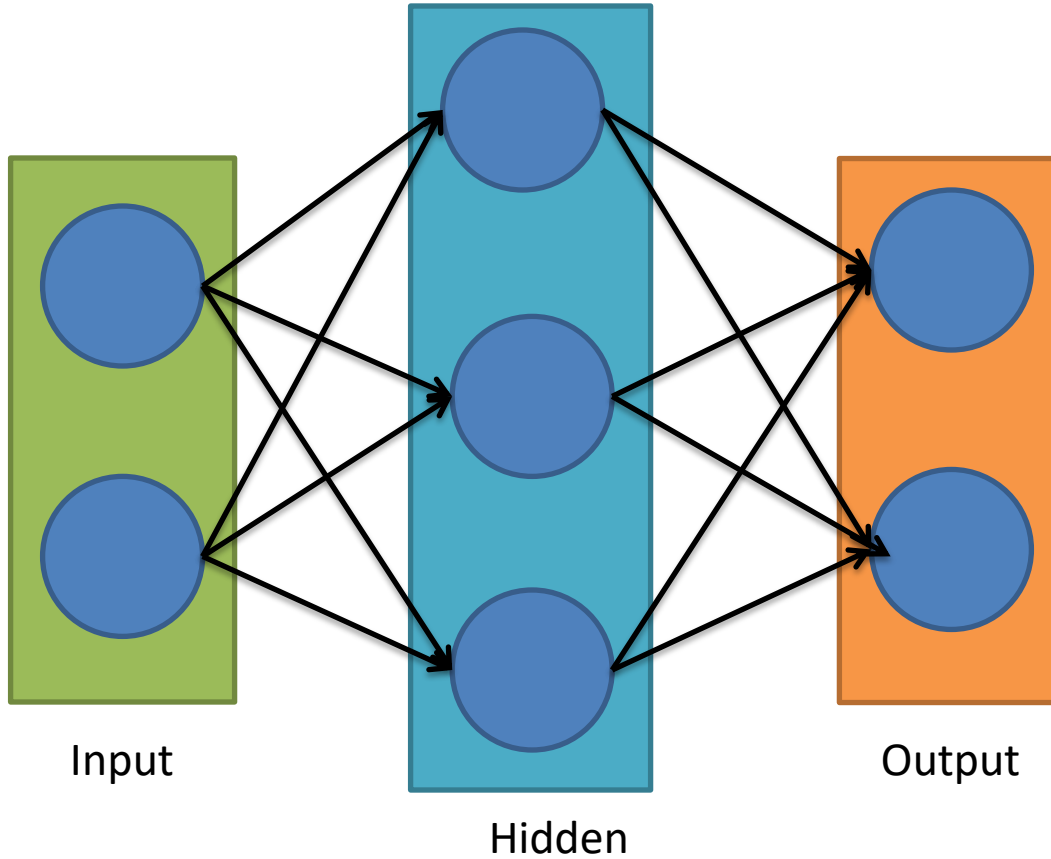


Introduction to Large Language Models (LLMs)

Neural Language Model Chapter 2



Feed Forward Neural Network



Optimizers

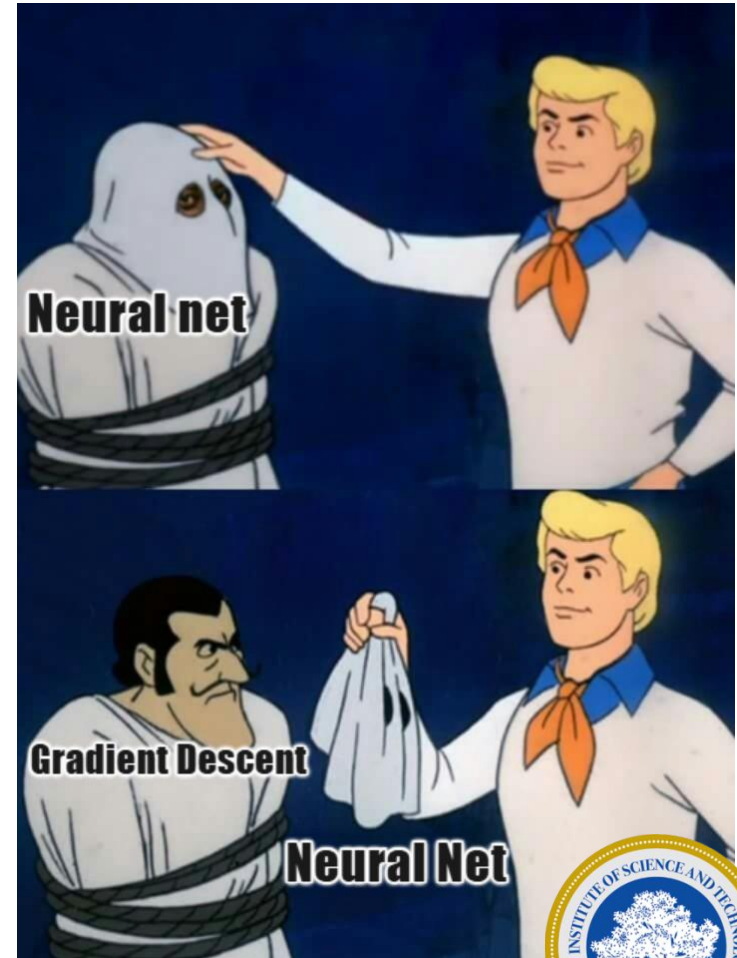
$$L = - \sum_{i=1}^2 t_i \log(p_i)$$
$$= - [t \log(p) + (1 - t) \log(1 - p)]$$

where t_i is the truth value taking a value 0 or 1 and p_i is the Softmax probability for the i^{th} class.

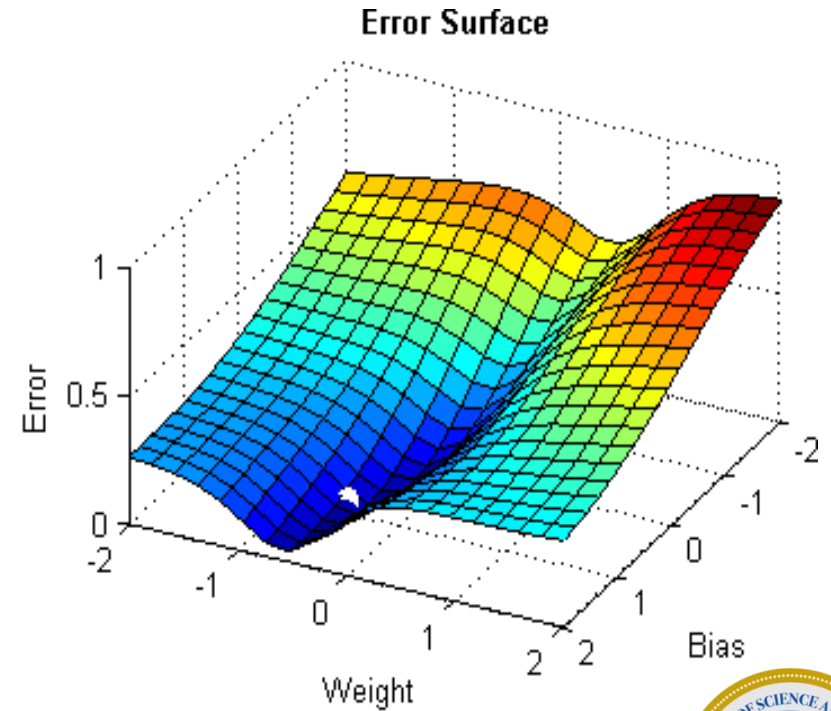
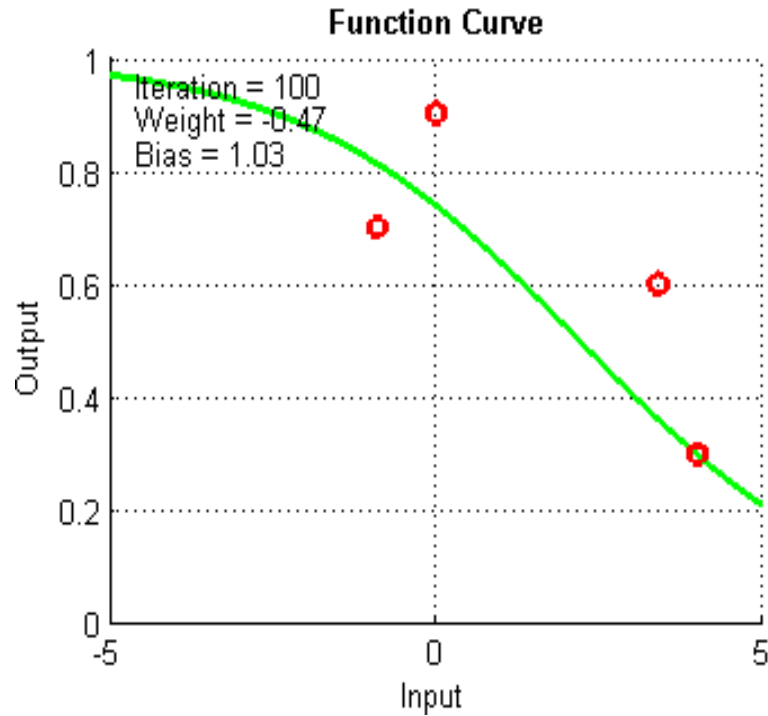
Weight Calculation

$$w = w - (\alpha \times (\text{Grad}(\text{loss})))$$

Gradient Optimizer



Gradient Descent



Fitting a function with gradient descent ([from Arvind Na](#))

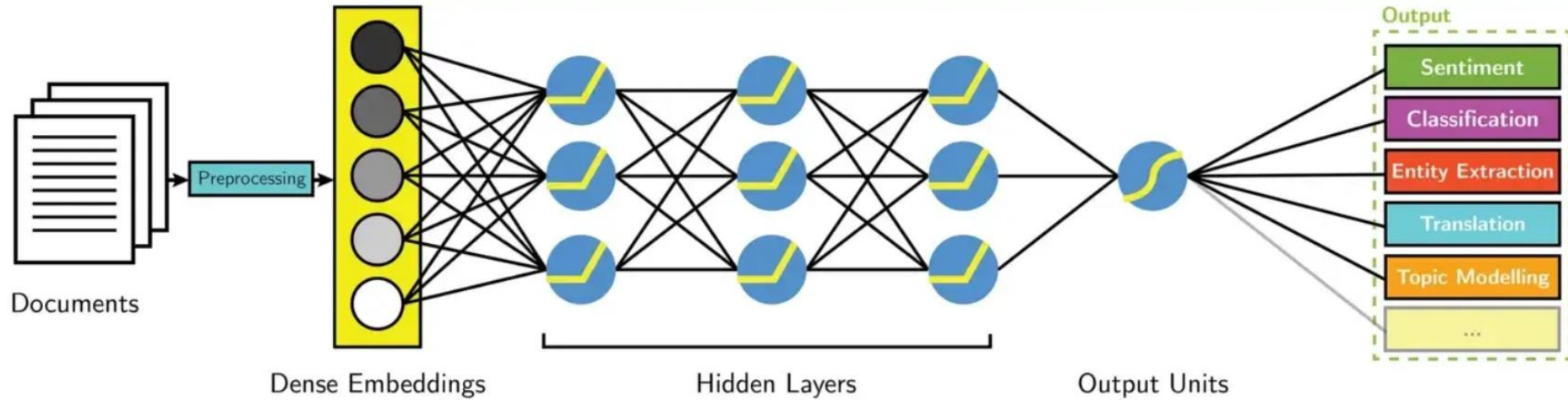


Feed Forward Neural Network Language Model (FFNLM)

- Let's start by breaking down the term. **“Feedforward”** describes how information moves through the model — it flows in one direction, from input to output, without looping back.
- **“Neural Network”** refers to the model's architecture, inspired by the human brain, consisting of layers of interconnected units (or "neurons") that process data collectively.
- **“Language Model”** simply means the model is built to understand and generate language-based data.



Feed Forward Neural Network Language Model (FFNLM)

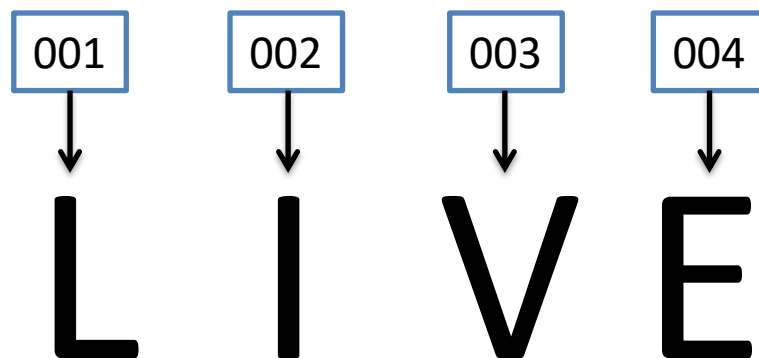


Source: Aylien

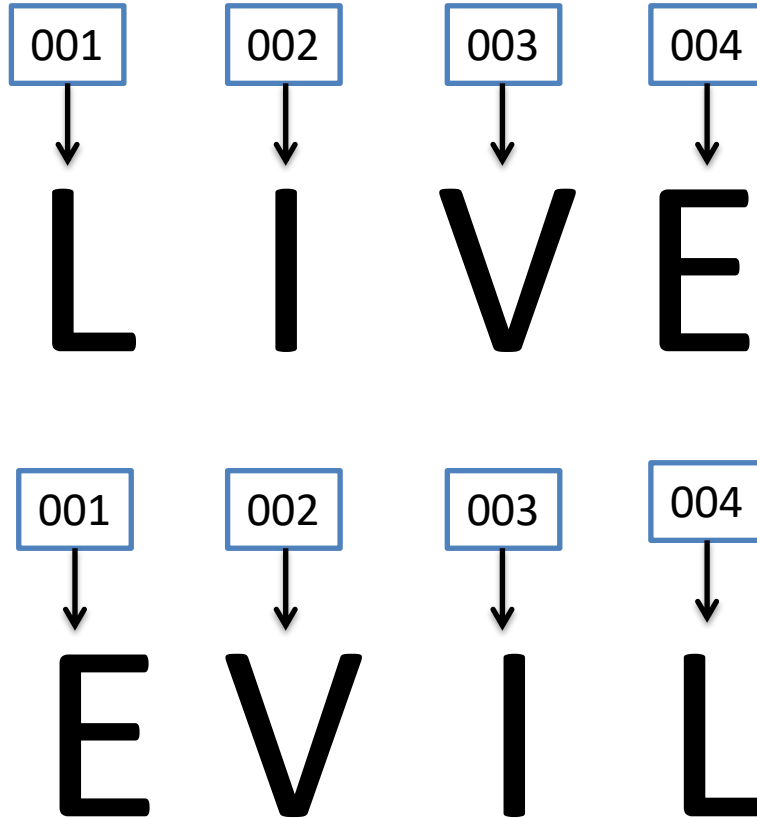
Embedding layer: This layer is responsible for converting the input words into a numerical representation called an embedding. The embeddings are learned during training and are used to represent the meaning of words in a continuous vector space.



Word Embedding



Word Embedding



Word Embedding

001
↓

002
↓

003
↓

004
↓

I love my dog



Word Embedding

001



I

002



love

003



my

004



dog

001



I

002



love

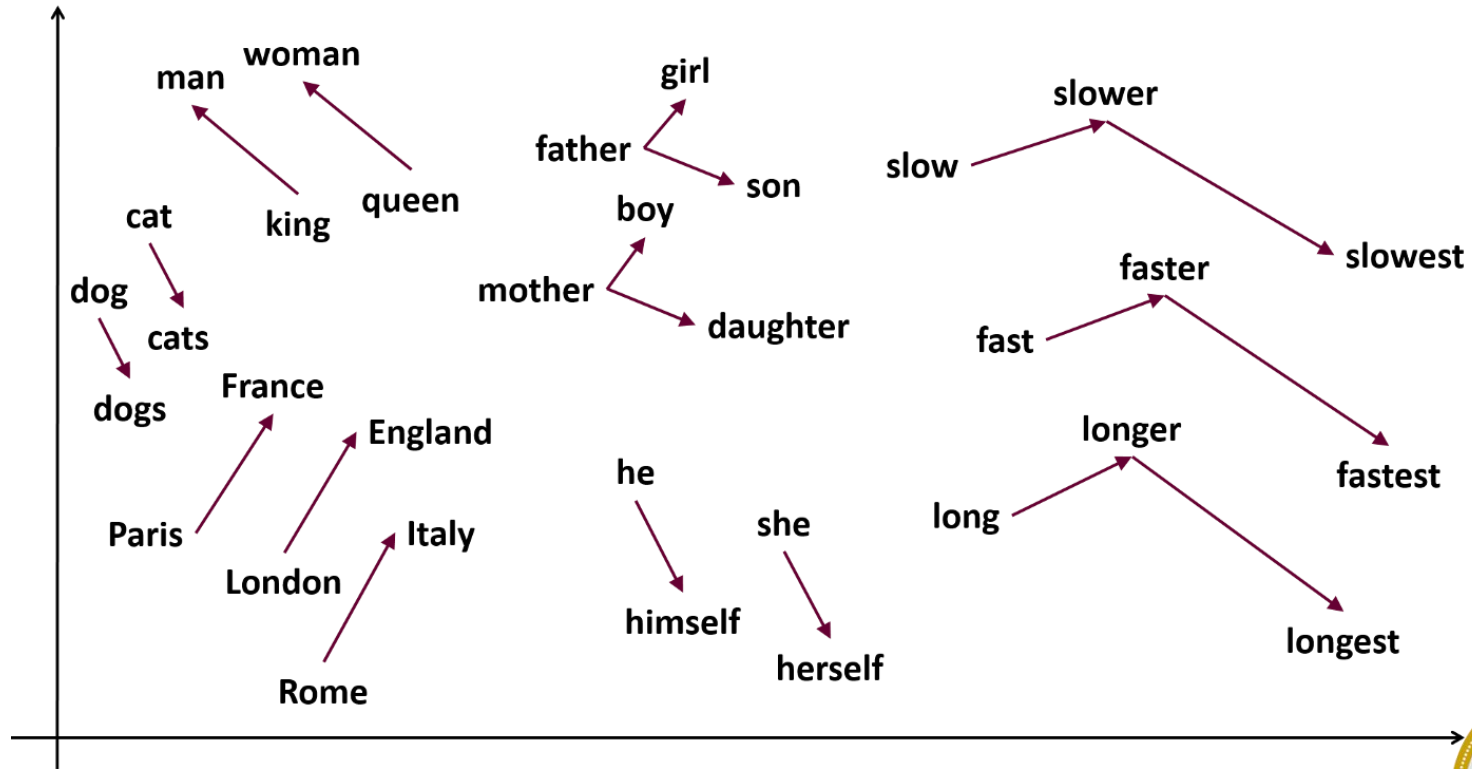
005



myself



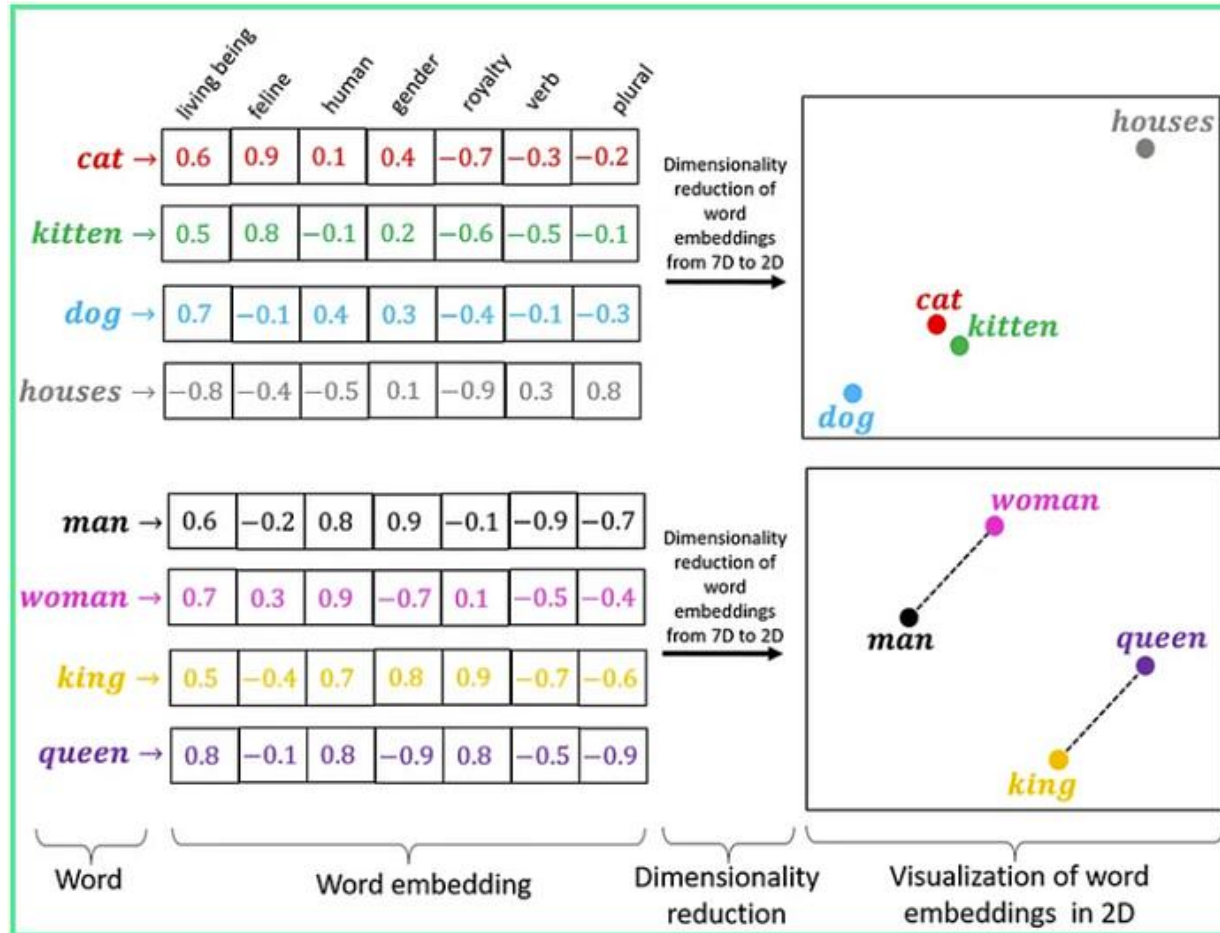
Word Embeddings



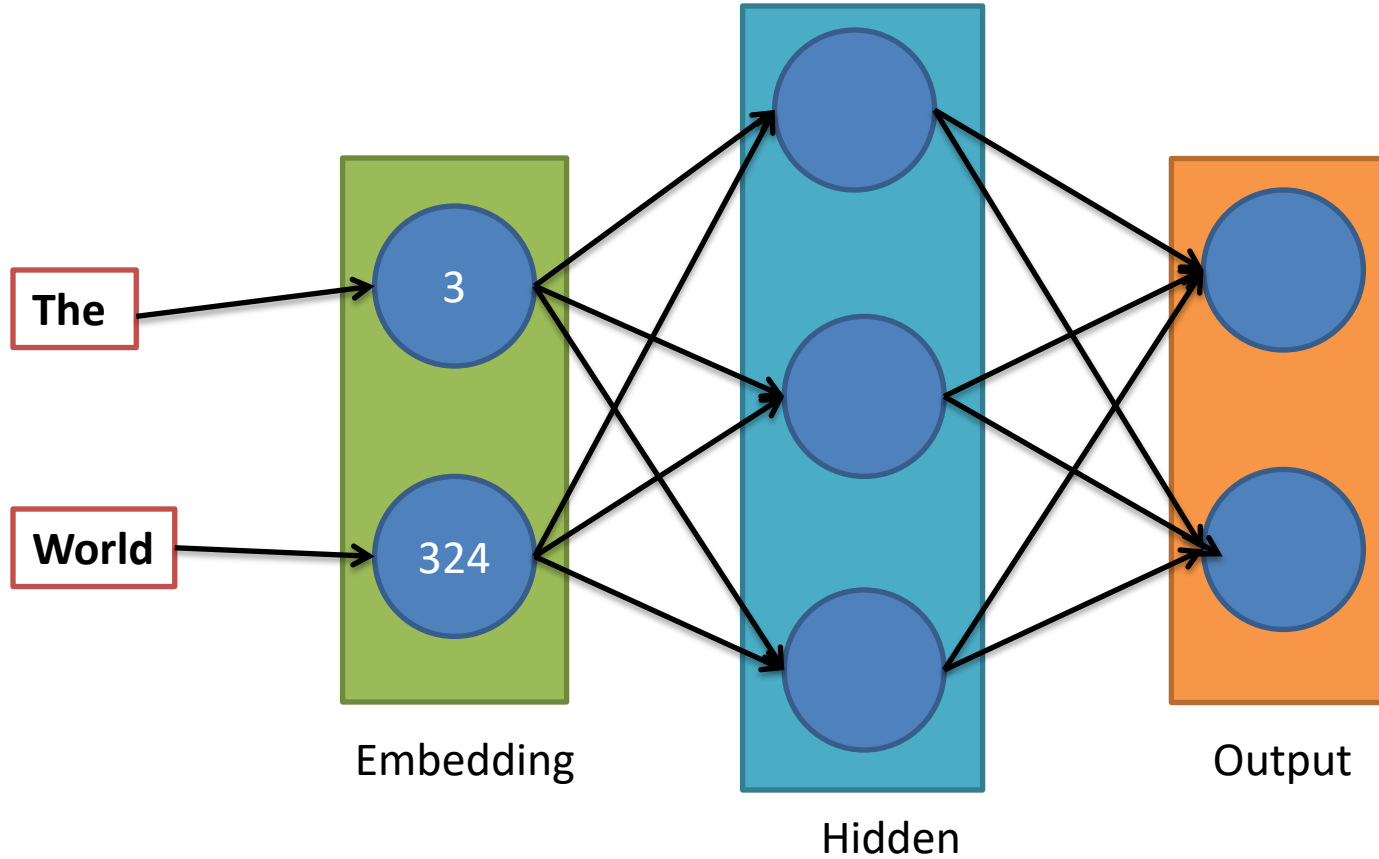
https://link.springer.com/chapter/10.1007/978-981-96-0029-8_5



Word Embeddings



FFNLM

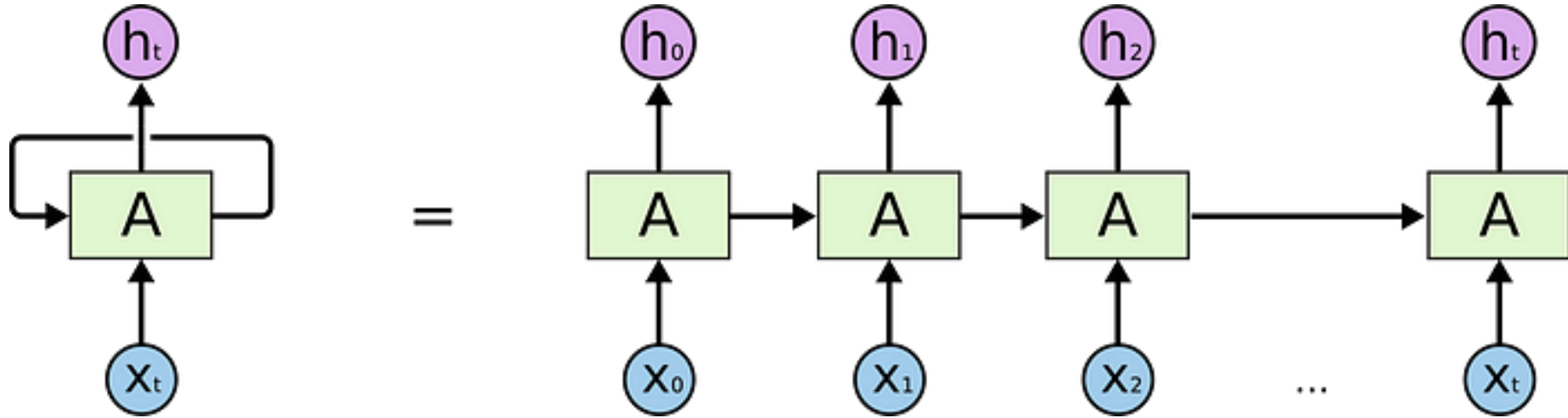


FFNLM

- Cannot handle Sequence
- Consider Present data
- No memory / Past Data



Recurrent Neural Network

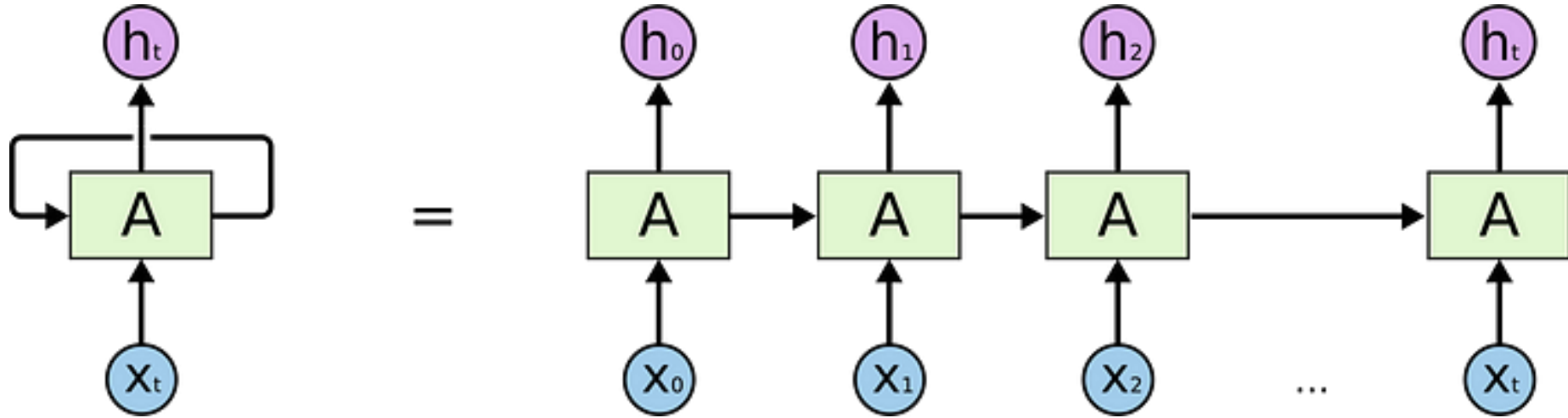


RNN

- One to Many – Image Captioning, Generation
- Many to One – Sentiment Analysis
- Many to Many – Translation, Activity recognition



RNN

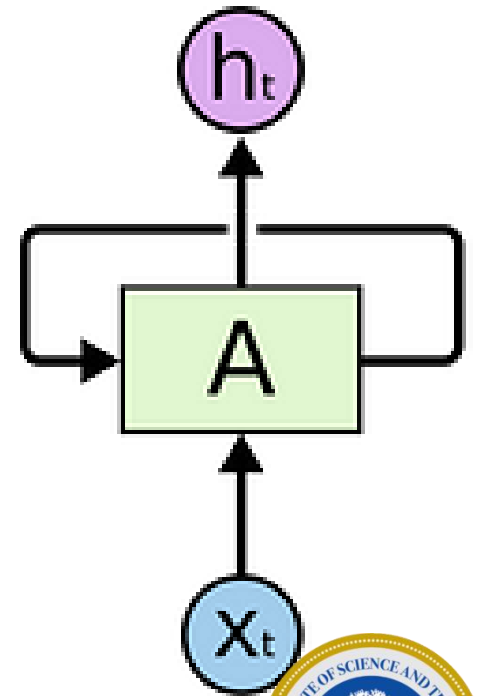


RNN

- Process a sequence of vectors x by applying a recurrence formula at every time step:

$$\boxed{H_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

New State Activation & Weight Old State Input Vector



RNN

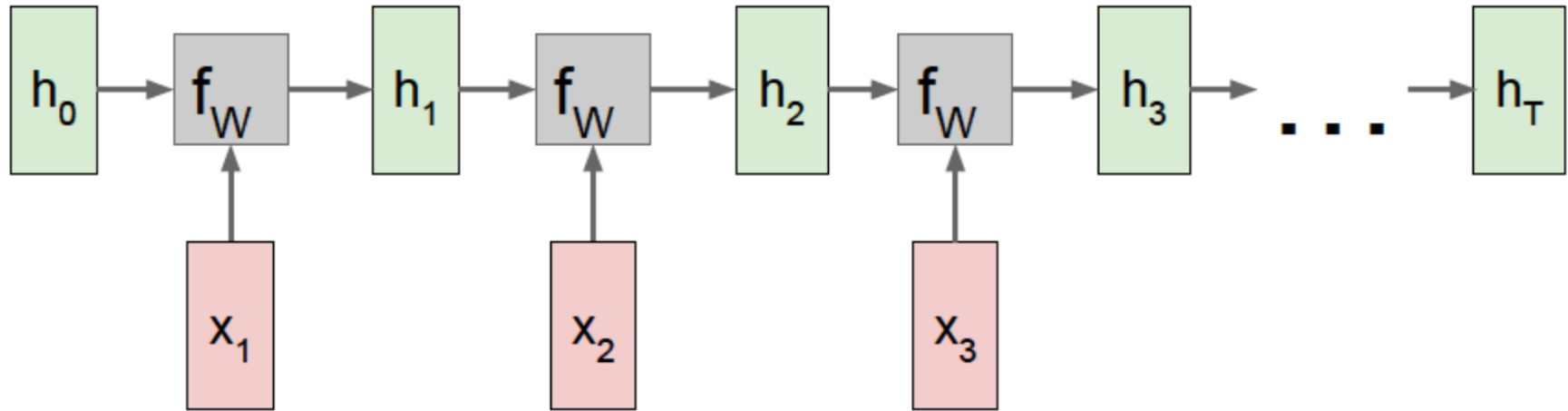
$$H_t = f_w(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

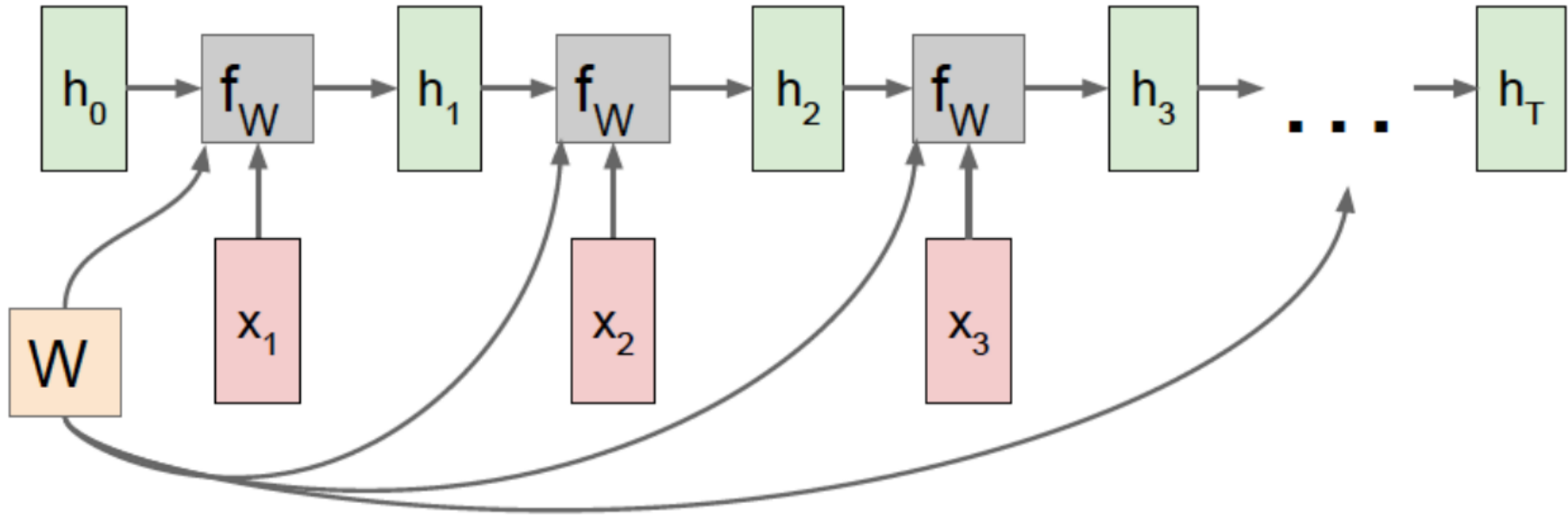
$$Y_t = W_{hy}h_t$$



RNN

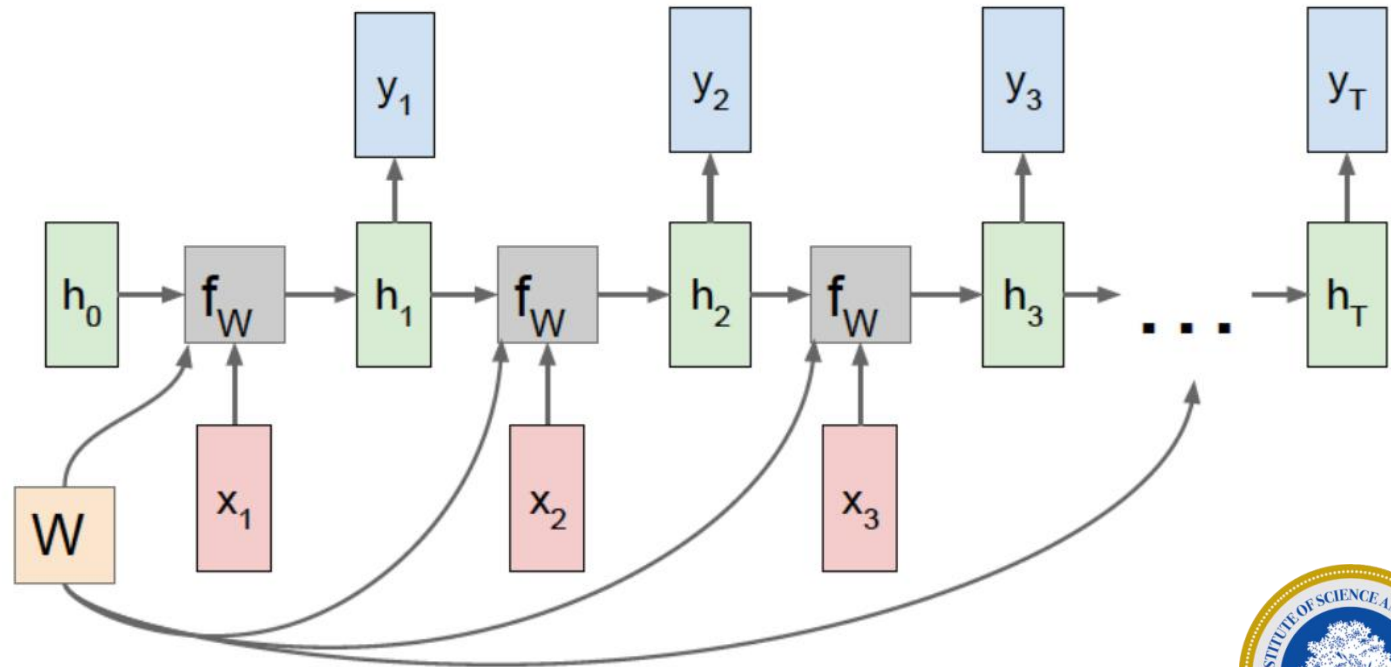


RNN



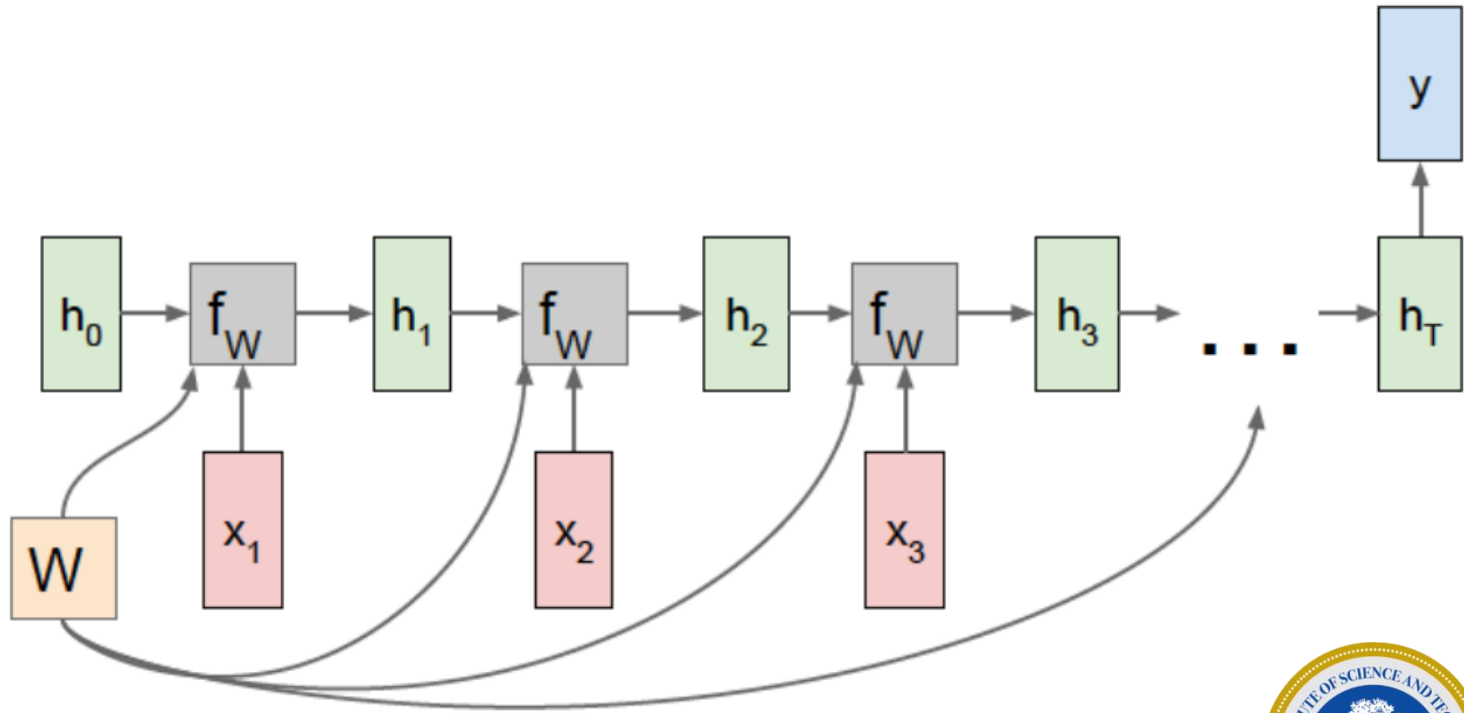
RNN

RNN: Computational Graph: Many to Many



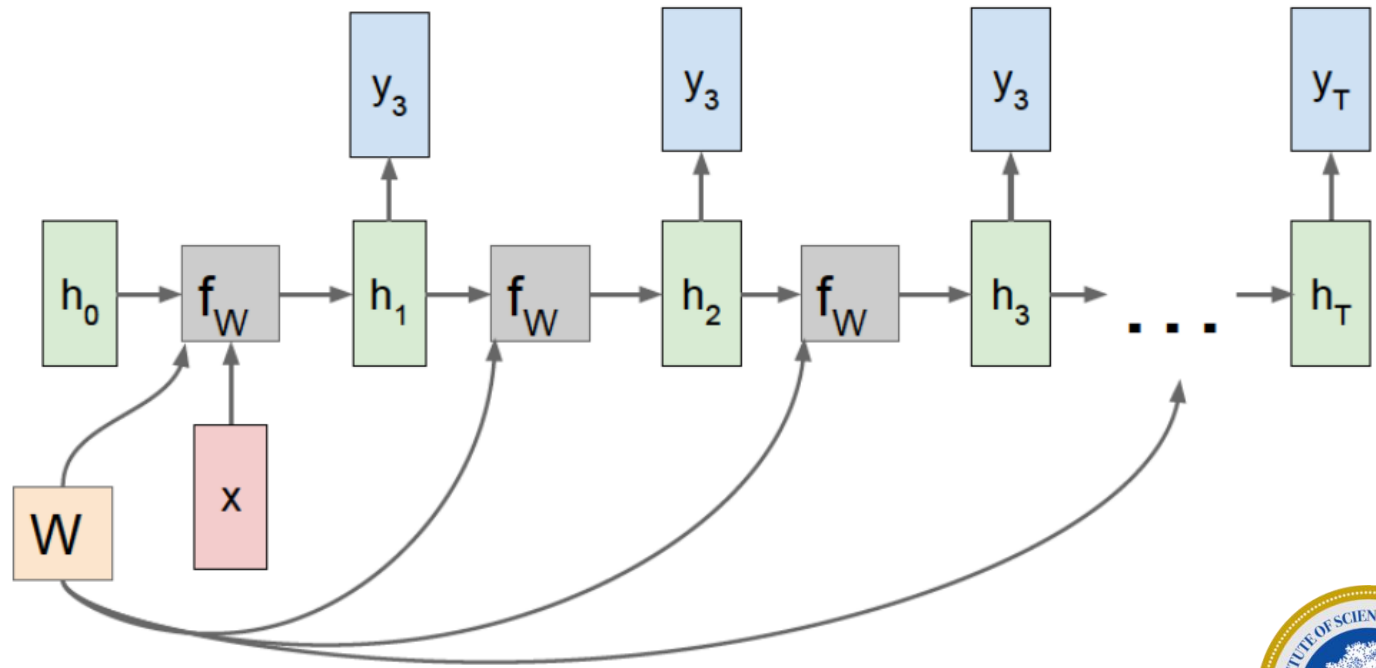
RNN

RNN: Computational Graph: **Many to One**

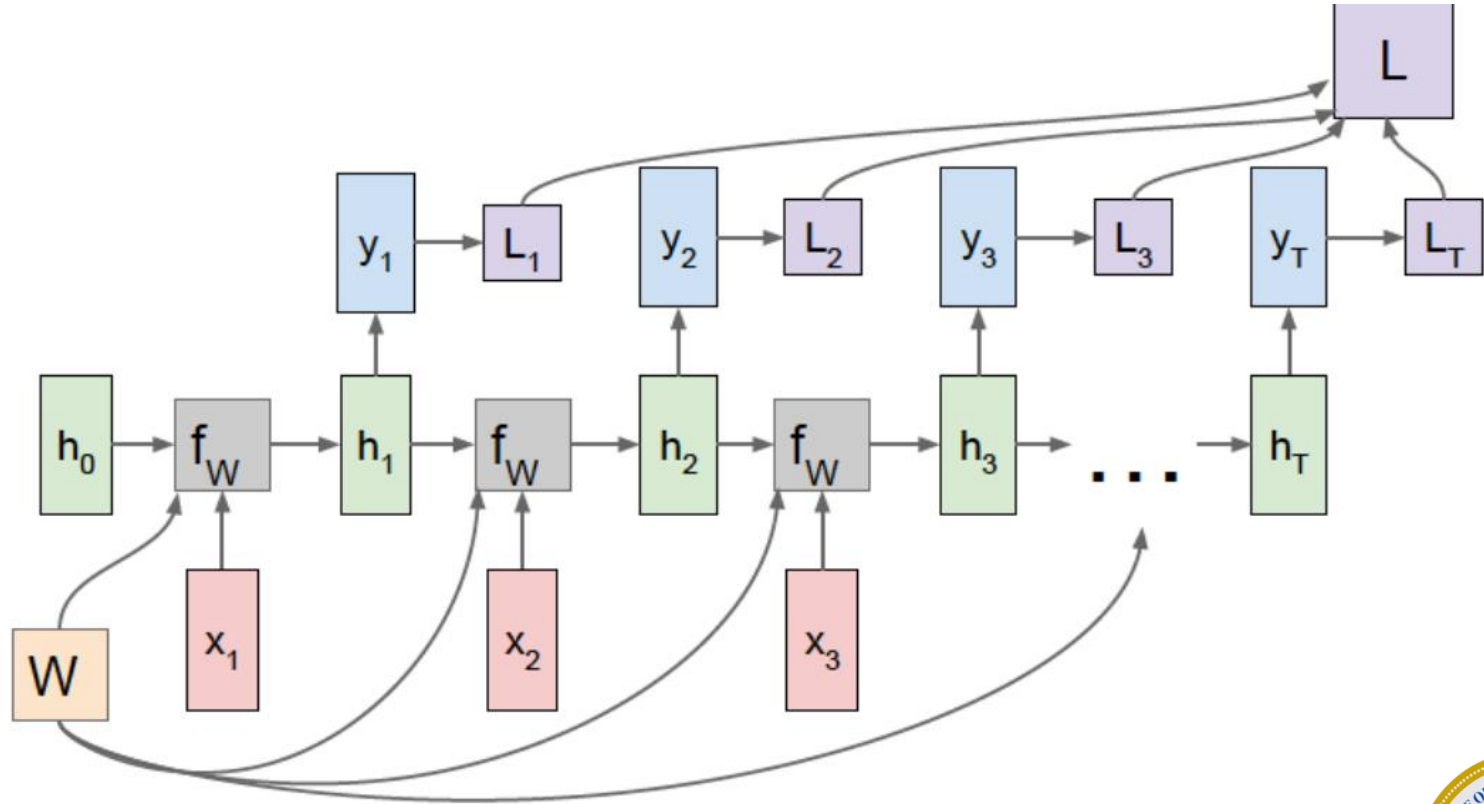


RNN

RNN: Computational Graph: **One to Many**

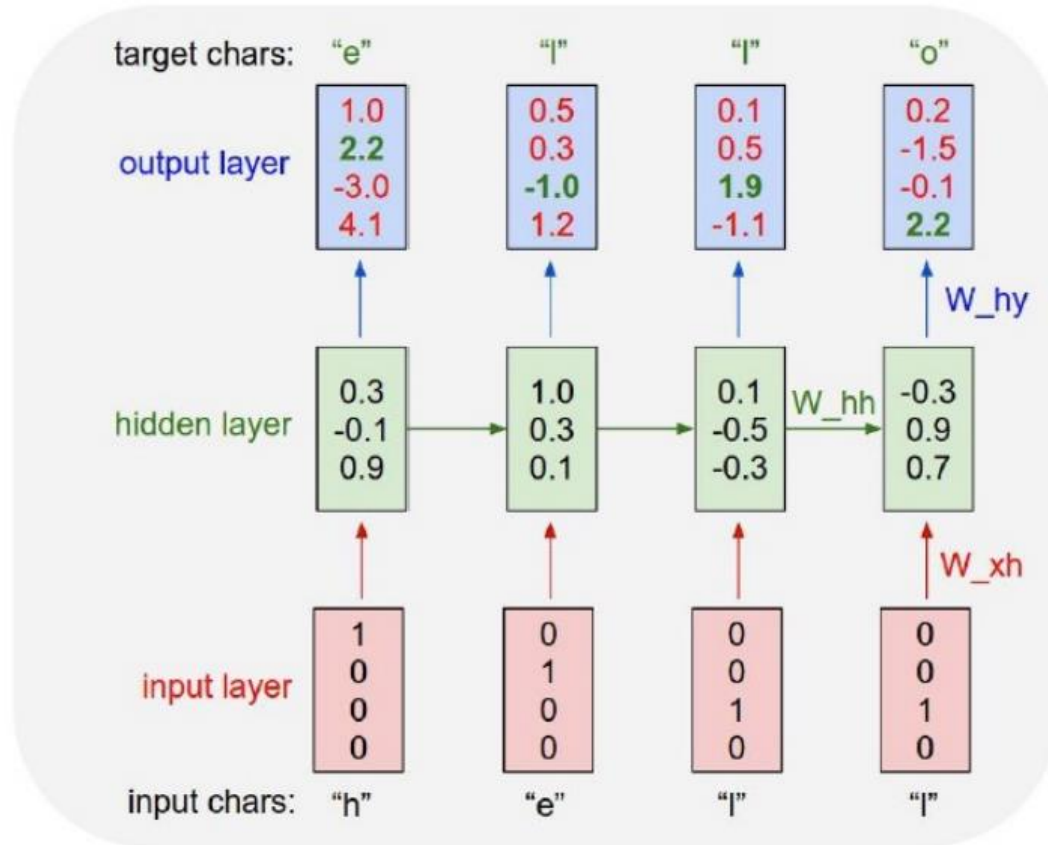


RNN

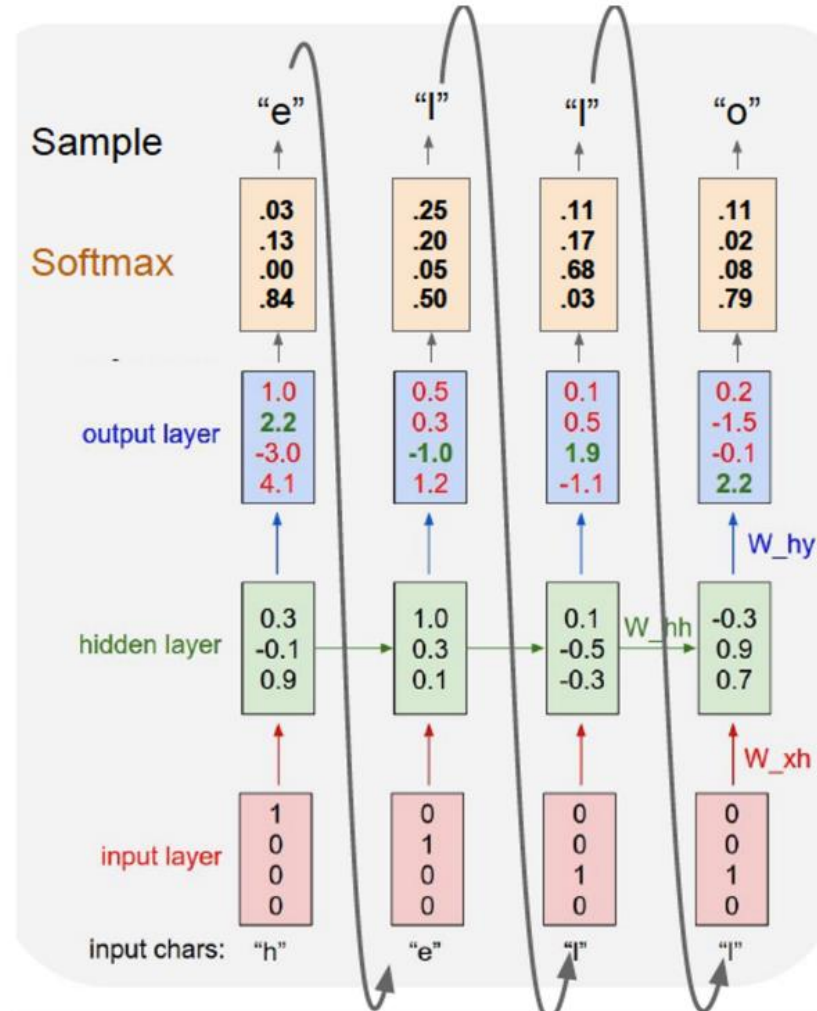


RNN

Example training sequence: "hello"



RNN



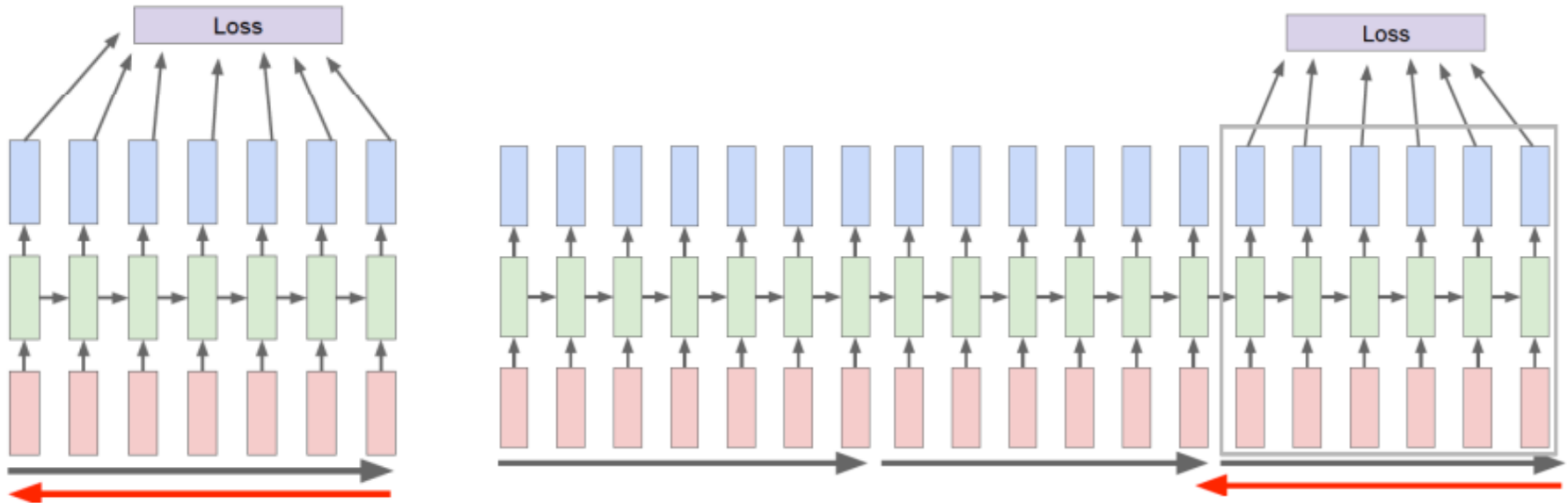
Back propagation through time

To train an RNN, we **unroll it across the required number of time steps** to match the shape of the input sequence. Then, we apply standard **automatic differentiation (autodiff)** for backpropagation.

First, we perform a **forward pass through the entire sequence** to calculate the total loss. Then, we do a **backward pass through the entire sequence** to compute gradients and update the model's weights.



Back propagation through time

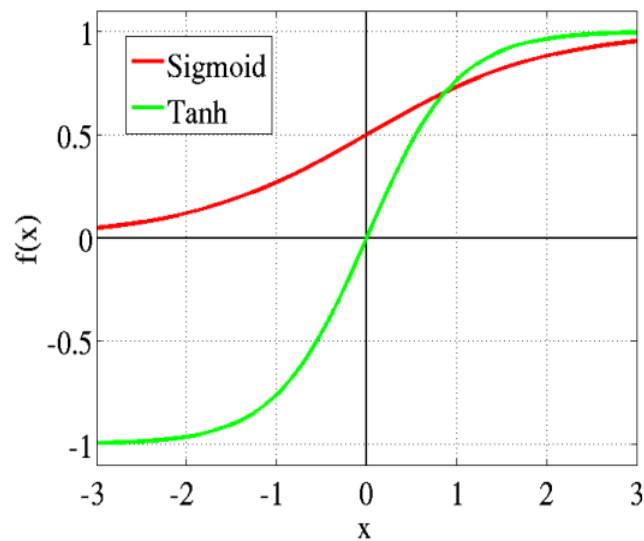


Gradient Problem

- Exploding Gradients
 - Identity Initialization
 - Truncated Back propagation
- Vanishing Gradients
 - Weights and activation function
 - LSTM

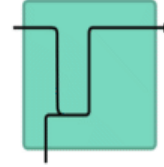
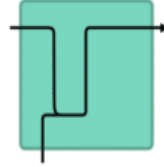
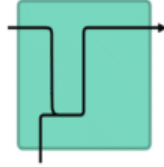
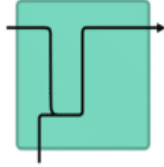


Tanh vs Sigmoid



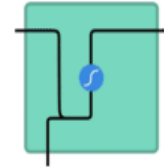
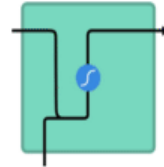
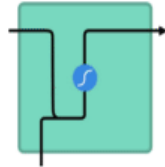
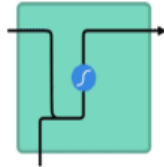
Without Tanh

5
0.01
-0.5



With Tanh

5
0.01
-0.5



Customers Review 2,491

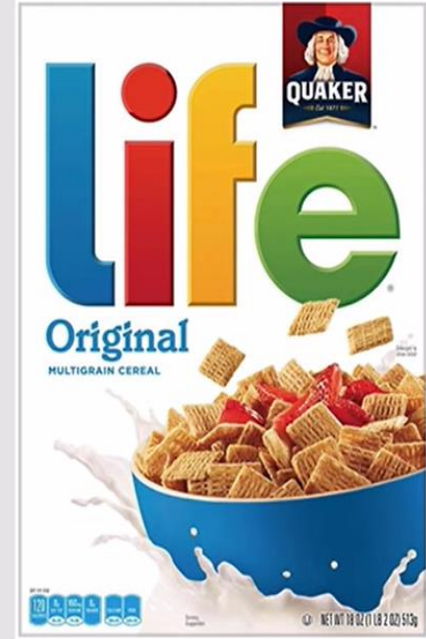


Thanos

September 2018

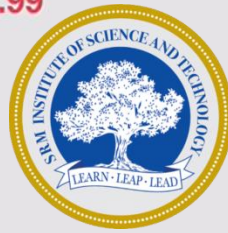
Verified Purchase

Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



A Box of Cereal

\$3.99

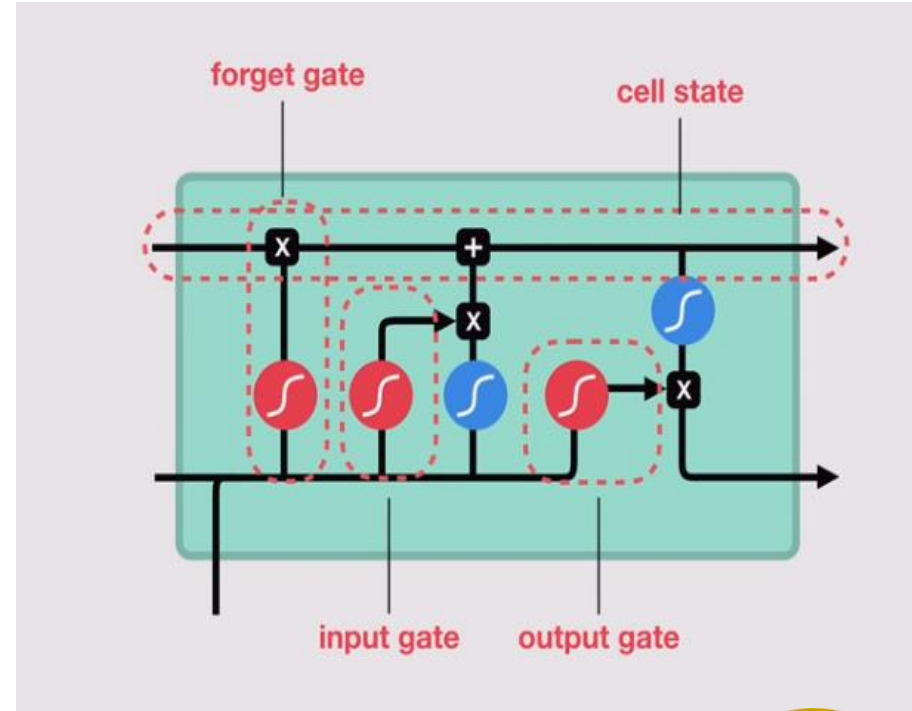


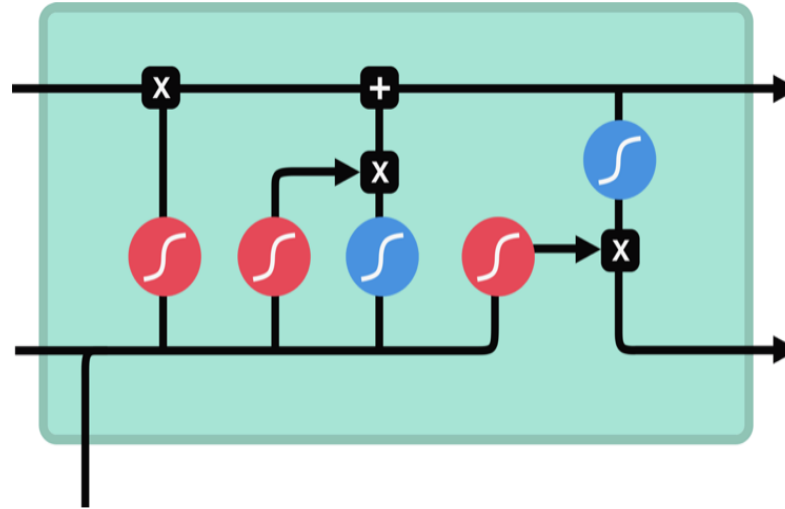
LSTM

Forget Gate: conditionally decides what information to discard from the unit.

Input Gate: conditionally decides which values from the input to update the memory state.

Output Gate: conditionally decides what to output based on input and the memory of the unit.





sigmoid



tanh



pointwise
multiplication



pointwise
addition



vector
concatenation



GRU – Gated Recurrent Unit

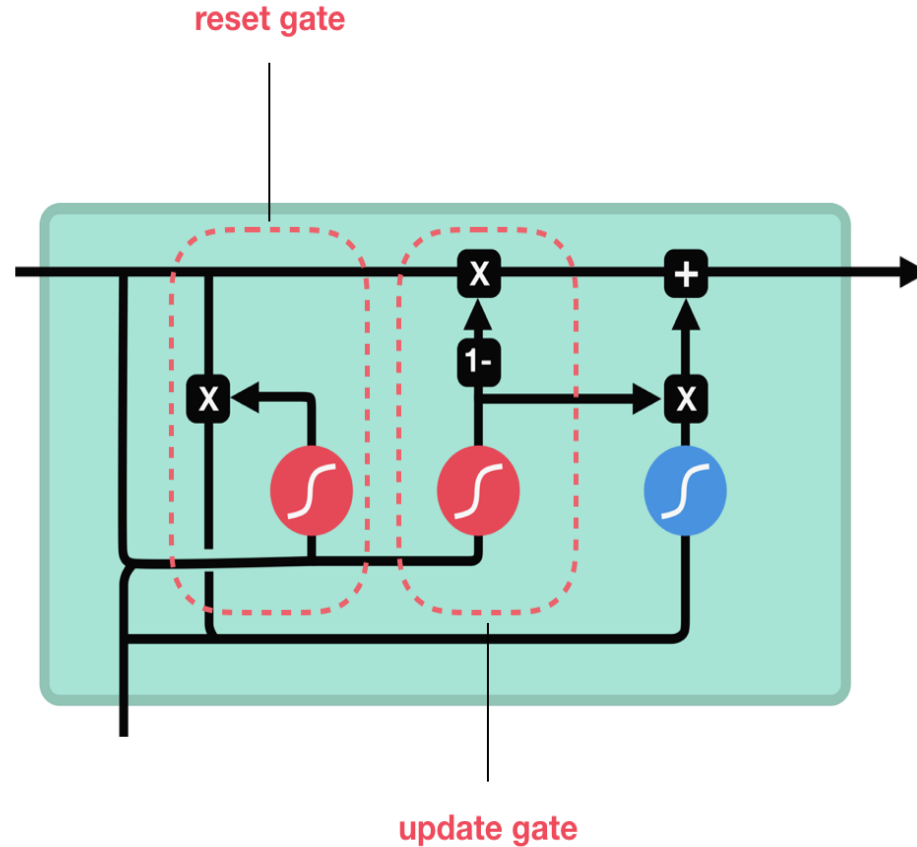


Image Captioning

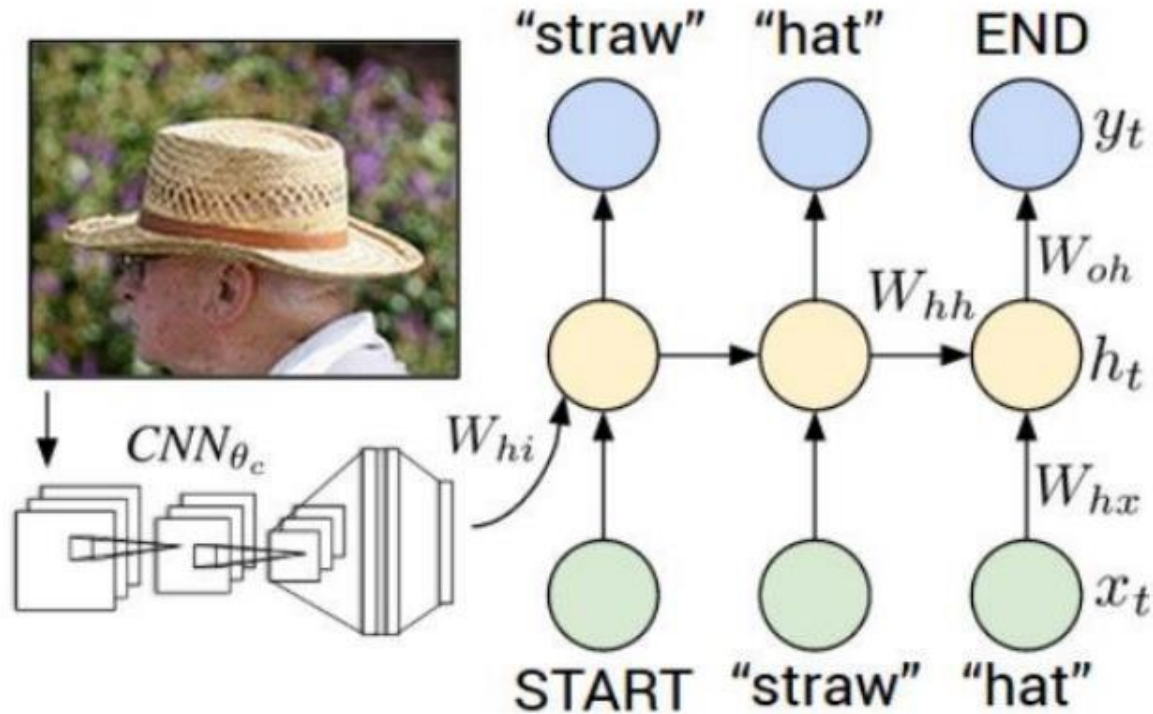


Figure from Karpathy et al, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015.



Image Captioning



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field

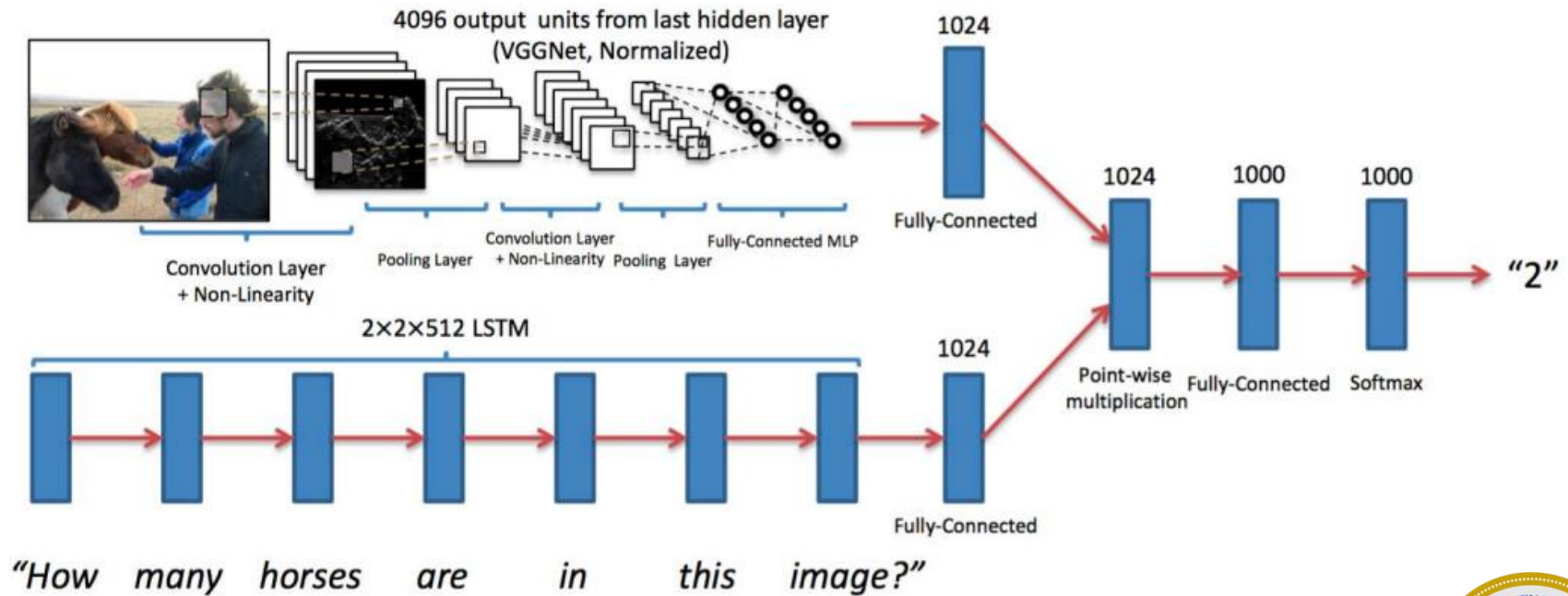


A man riding a dirt bike on a dirt track

<https://github.com/karpathy/neuraltalk2>



Visual QA



Still Sequential and takes more time to learn

Also cannot handle a very long text say 1000 words

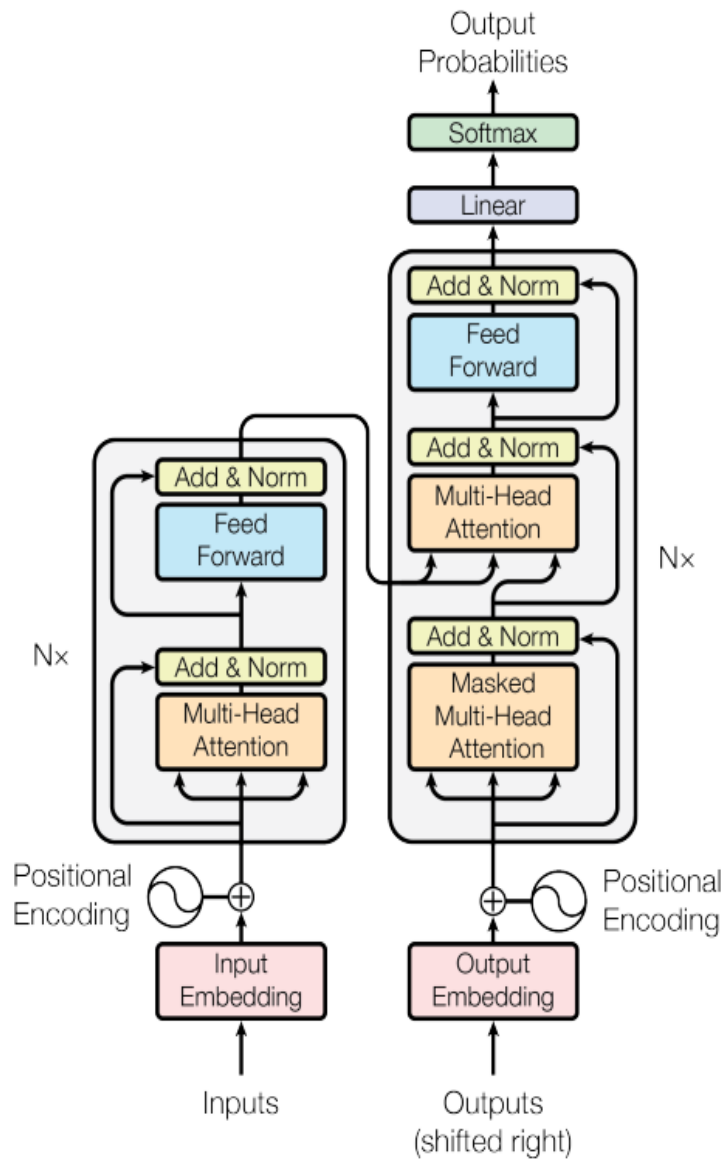
Context !



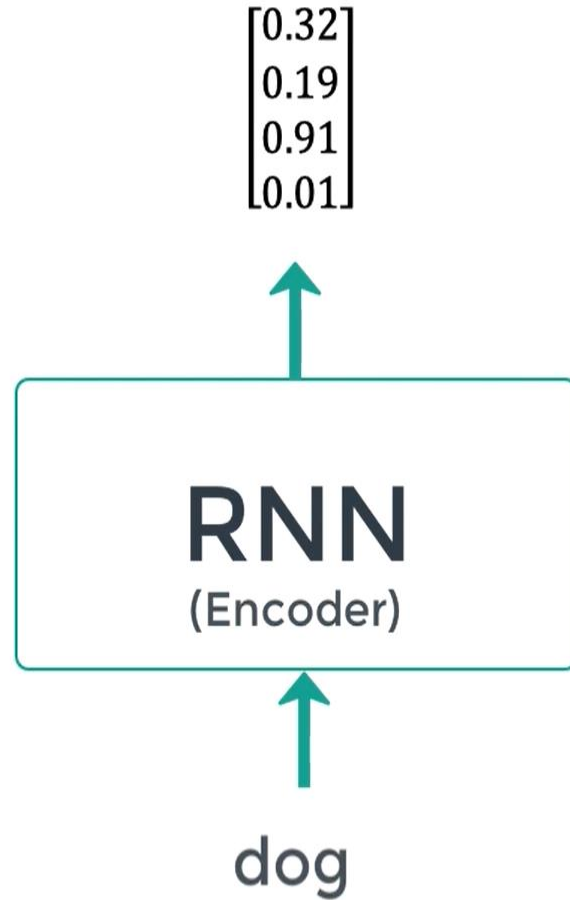
Transformer – Attention is all you Need

- Why use RNN if you need only the attention to take care of the problem.
- This to works on same idea of encoding and decoding
- But this time all the data are encoded at the same time

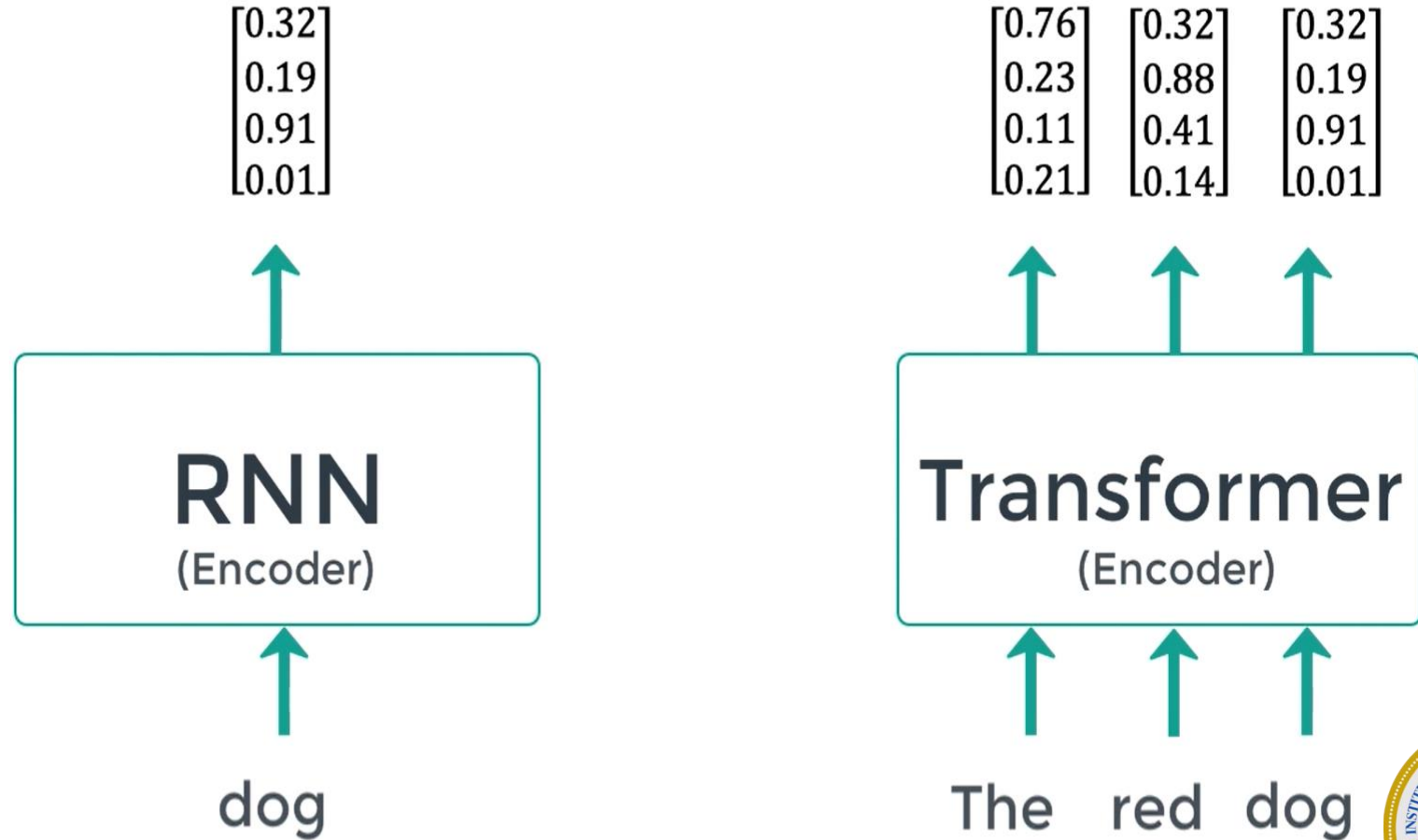




English-French Translation



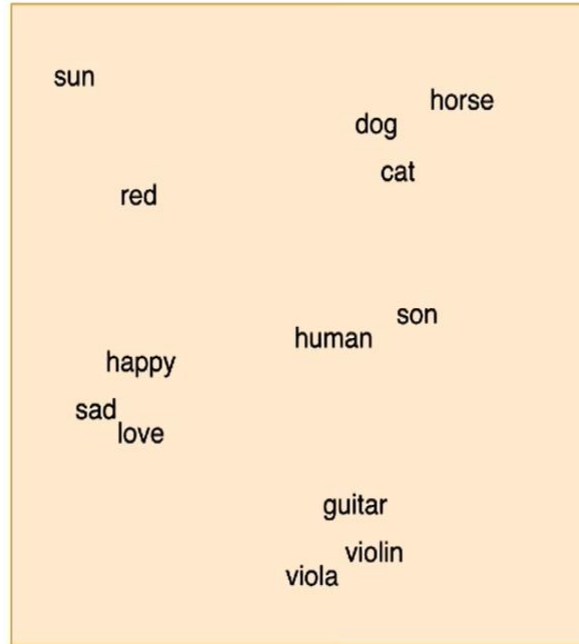
English-French Translation



Transformer Components

Input Embedding

dog


$$\begin{bmatrix} 0.37 \\ 0.99 \\ 0.01 \\ 0.08 \end{bmatrix}$$

AJ's **dog** is a cutie

AJ looks like a **dog**



Transformer Components

Positional Encoder :vector that gives context based on position of word in sentence

AJ's **dog** is a cutie → Position 2

AJ looks like a **dog** → Position 5



Transformer Components

Positional Encoder :vector that gives context based on position of word in sentence

AJ's **dog** is a cutie → Position 2

AJ looks like a **dog** → Position 5

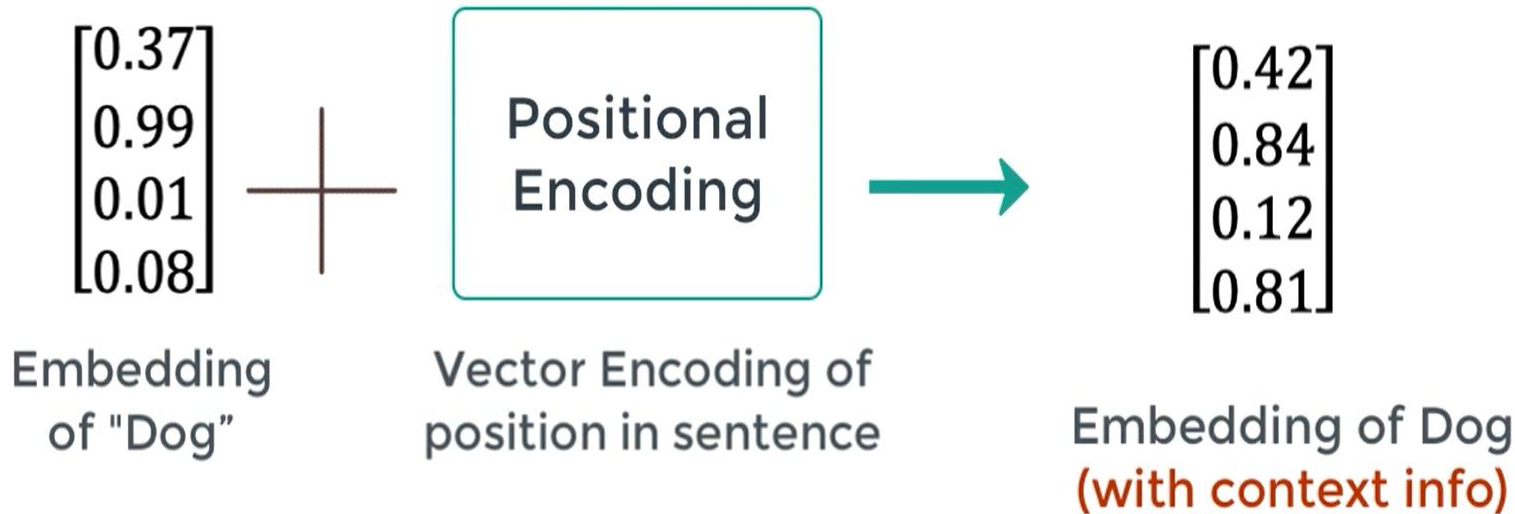
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



Transformer Components

Positional Encoder :vector that gives context based on position of word in sentence

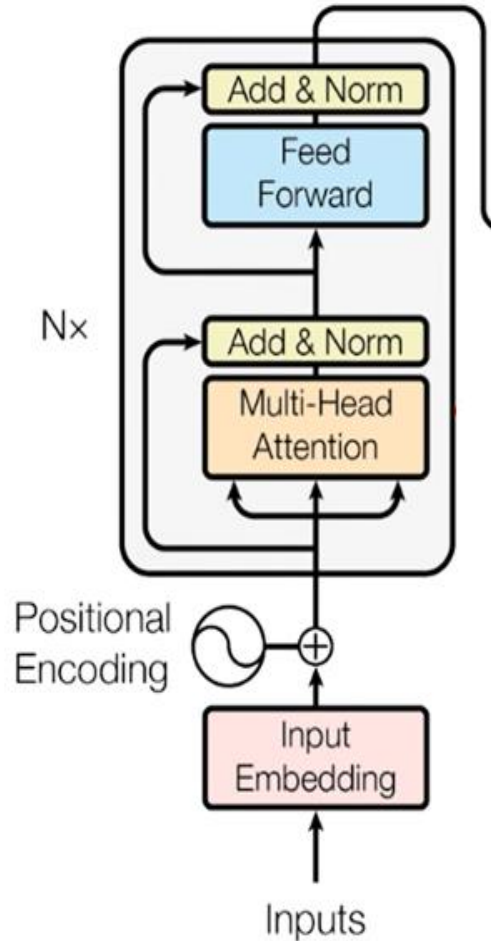


$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$



Transformer Components

Encoder Block



Transformer Components

Attention : What part of the input should we focus?

The → The big red dog
big → The big red dog
red → The big red dog
dog → The big red dog



Transformer Components

Attention : What part of the input should we focus?

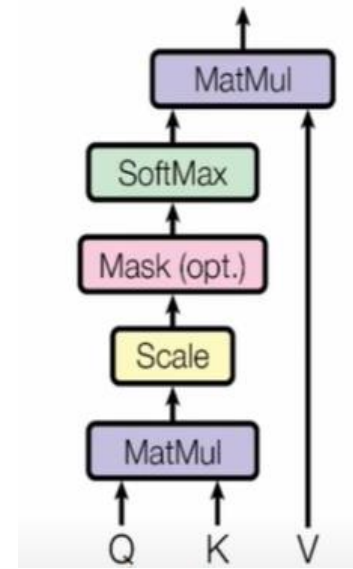
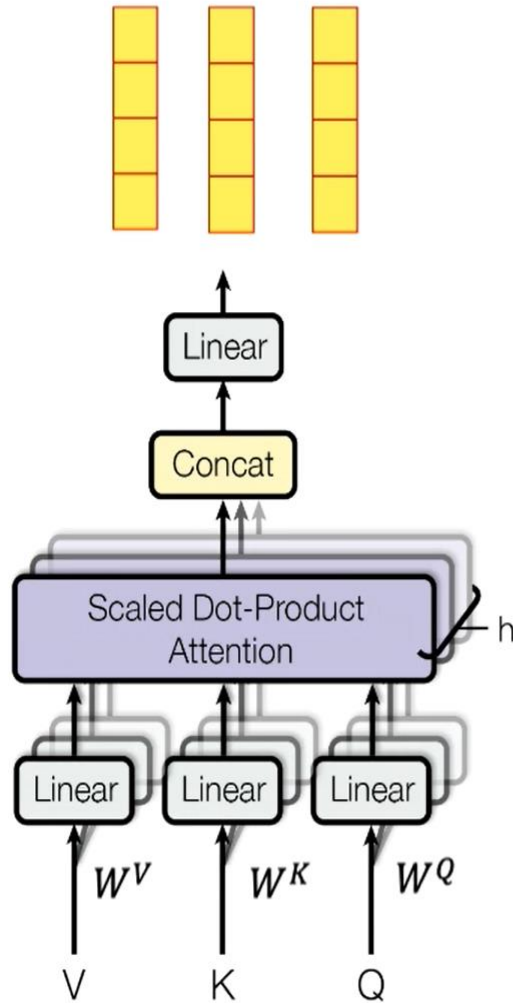
Focus

The	→	The big red dog
big	→	The big red dog
red	→	The big red dog
dog	→	The big red dog

Attention Vectors

[0.71	0.04	0.07	0.18]	^T
[0.01	0.84	0.02	0.13]	^T
[0.09	0.05	0.62	0.24]	^T
[0.03	0.03	0.03	0.91]	^T





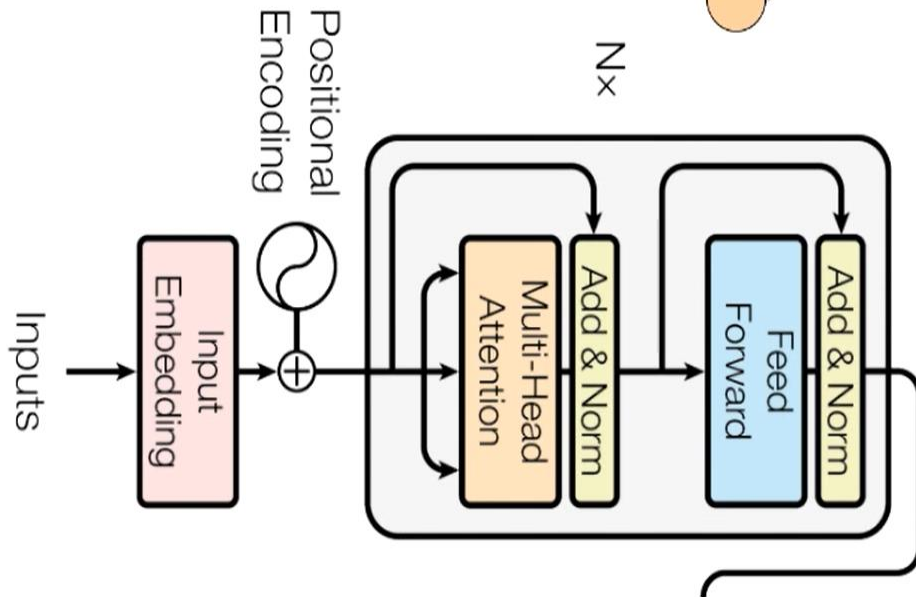
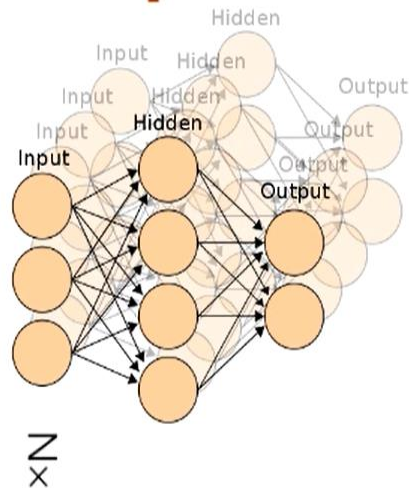
$$Z = \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{\text{Dimension of vector } Q, K \text{ or } V}}} \right) \cdot V$$



Transformer Components

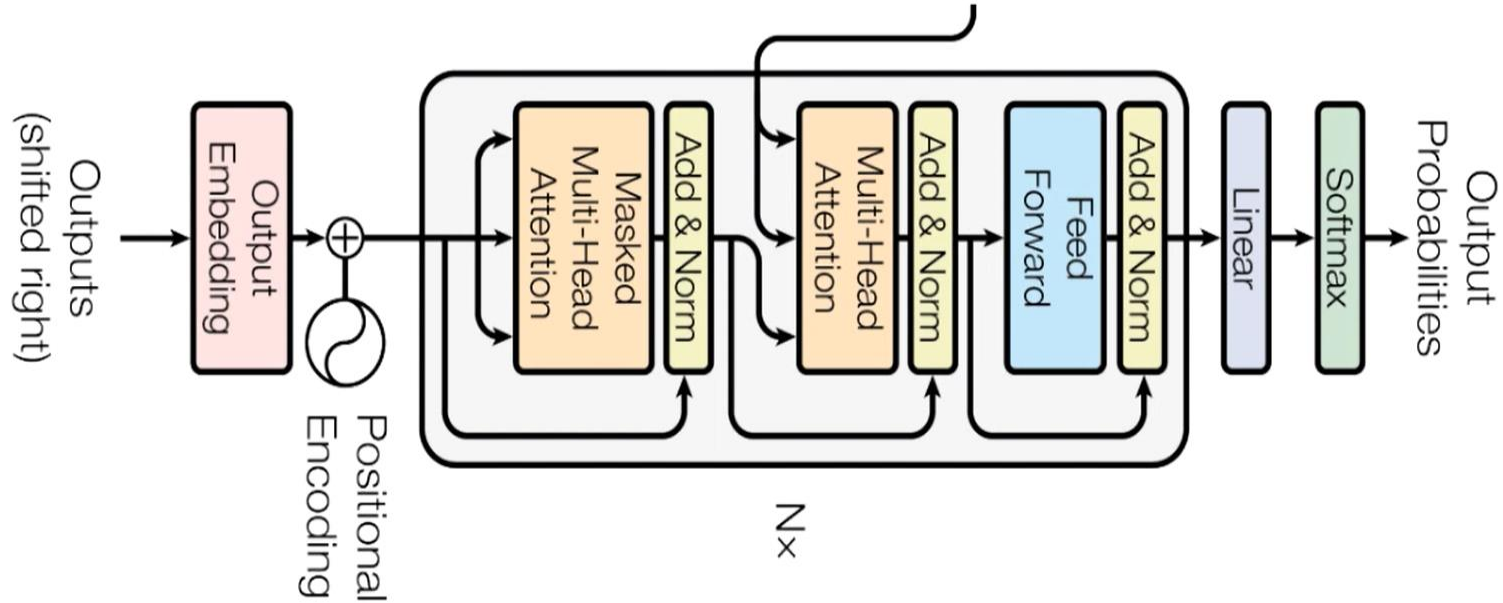
The big red dog

$\begin{bmatrix} 0.71 \\ 0.04 \\ 0.07 \\ 0.18 \end{bmatrix}$



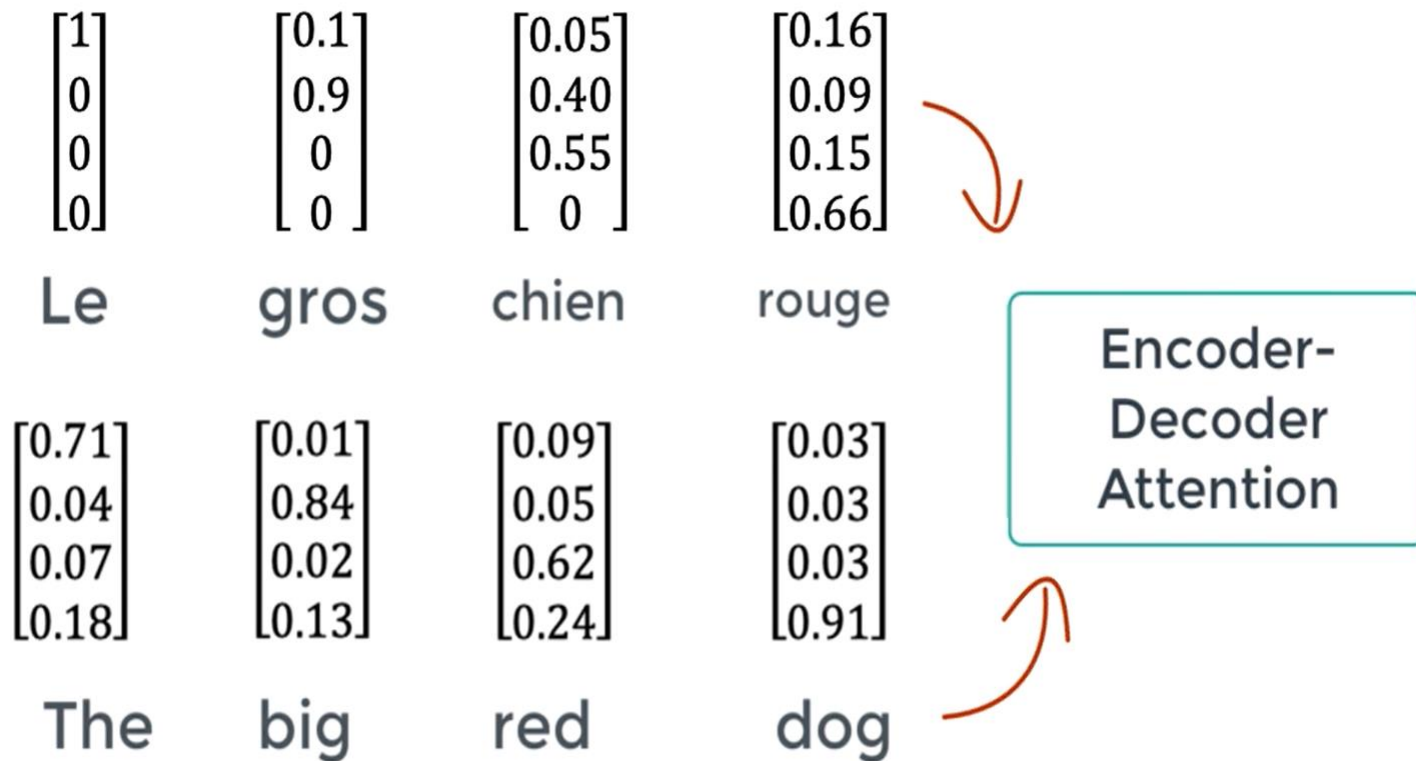
Transformer Components

Decoder



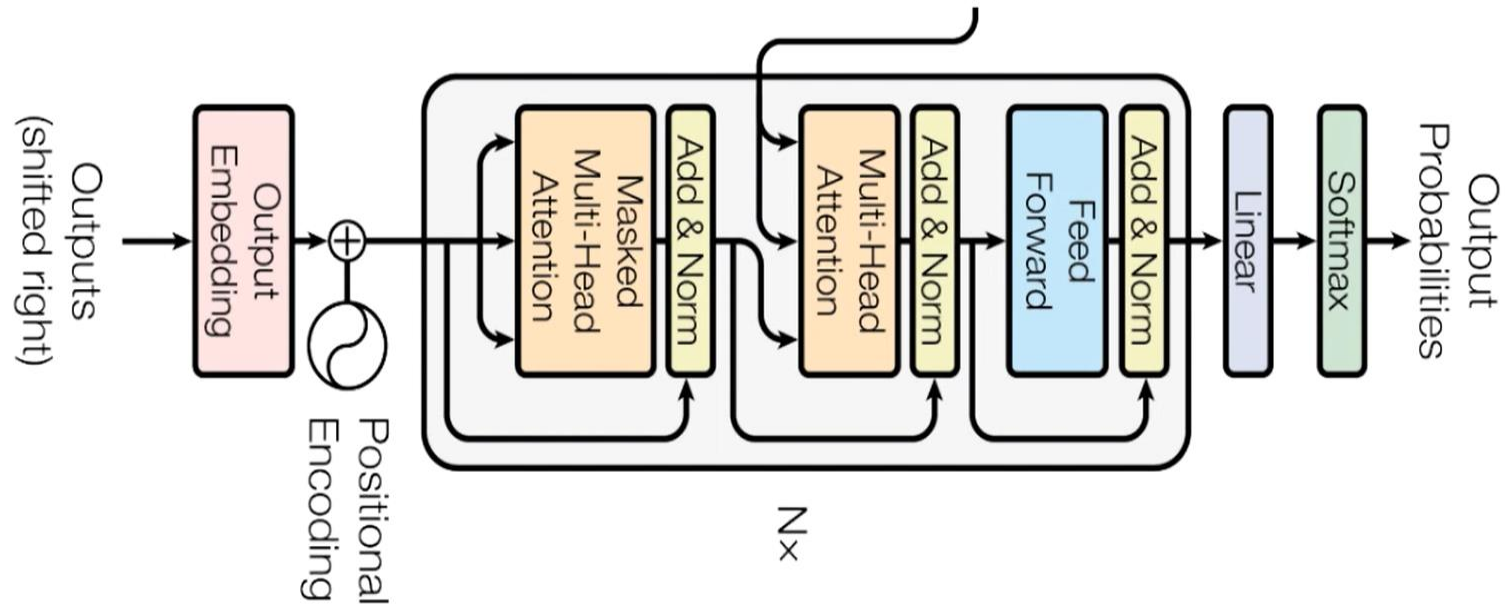
Transformer Components

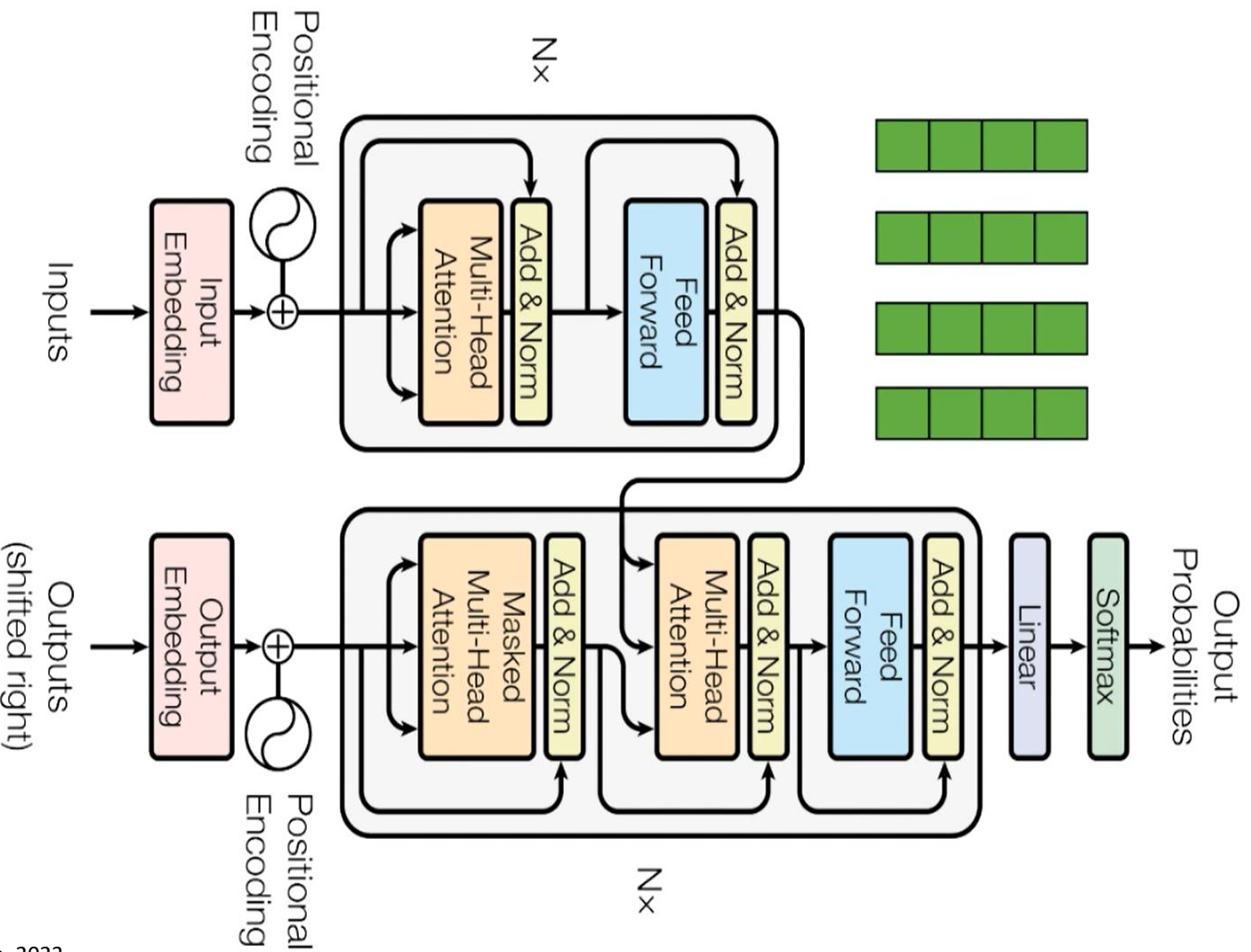
Decoder



Transformer Components

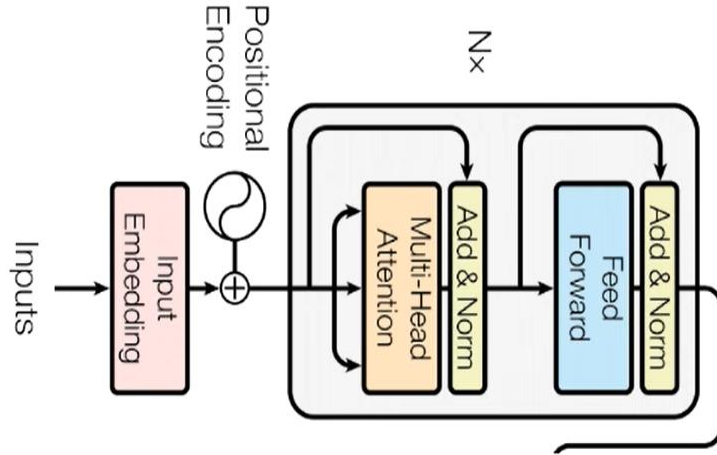
Decoder



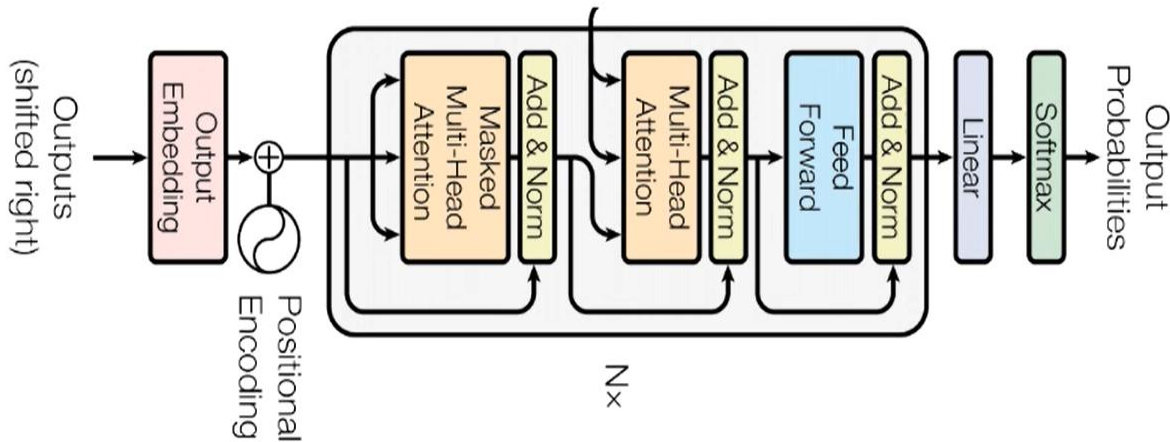


The
big
red
dog

Transformer Flow



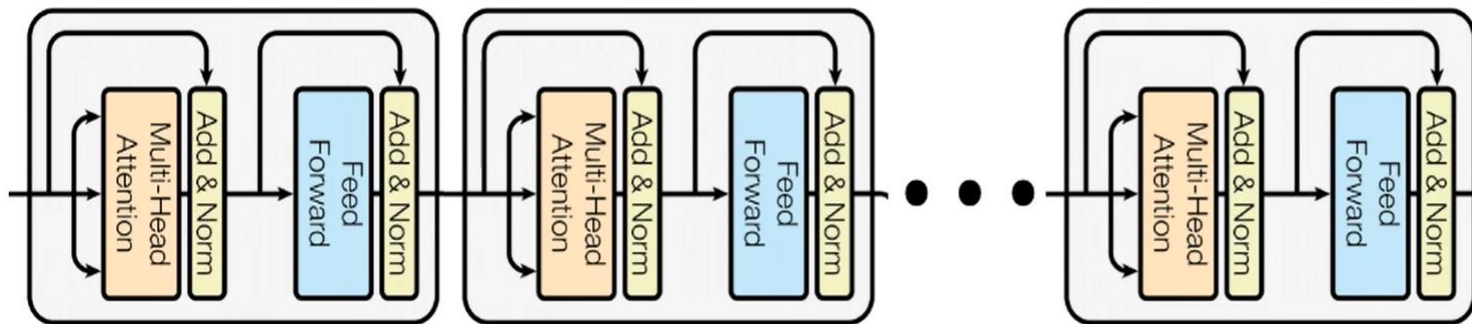
What is English? What is context?
What is language!



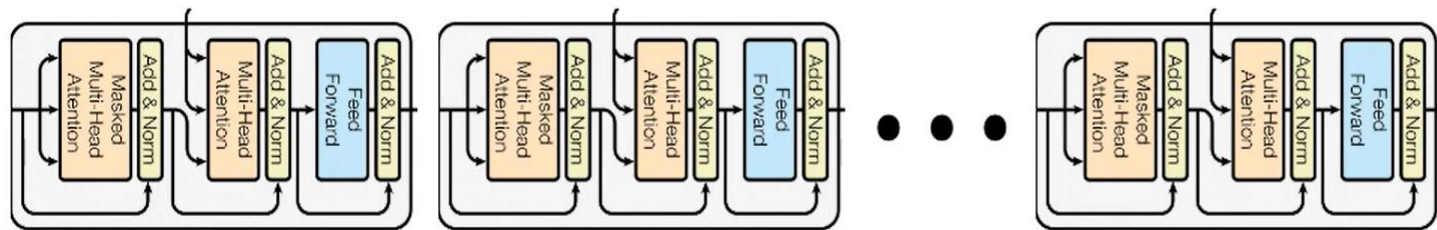
How to map English words to French words?
What is language!



Transformer Flow



BERT



GPT



End of Slide

THANK YOU

